```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <semaphore.h>
#include <errno.h>

#define SHM_NAME "/arche_noah_shm"
#define SEM_MUTEX "/arche_noah_mutex"
#define SEM_FULL "/arche_noah_full"
#define N 10

struct shared_data
{
    int bookings[N];
    int count;
};

void sem_wait_intr(sem_t *sem)
{
    while (sem_wait(sem) == -1)
    {
        if (errno != EINTR)
        {
            perror("sem_wait");
            exit(1);          alles schliessen nicht vergessen, ansonsten signal richtig behandelt
        }
    }
}

int main()
{
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open");
        exit(1);
    }

    ftruncate(shm_fd, sizeof(struct shared_data));

    struct shared_data *data = mmap(NULL, sizeof(*data),
                                    PROT_READ | PROT_WRITE,
                                    MAP_SHARED, shm_fd, 0);

    sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
    sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);

    if (mutex == SEM_FAILED || full == SEM_FAILED)
    {
        perror("sem_open");   alles schliessen nicht vergessen (-1)
        exit(1);
    }

    printf("Master waites for complete Buchungsliste...\n");

    sem_wait_intr(full);

    sem_wait_intr(mutex);
    printf("Buchungsliste complete:\n");
    for (int i = 0; i < N; i++)
    {
        printf("Space %d: Client %d\n", i, data->bookings[i]);
    }
    sem_post(mutex);

```

```
    munmap(data, sizeof(*data));
    close(shm_fd);

    shm_unlink(SHM_NAME);
    sem_unlink(SEM_MUTEX);
    sem_unlink(SEM_FULL);

    return 0;
}
```